Table of contents

Introduction

Background

I am currently enrolled in STATS200Q: Philosophical Foundations of Statistics with Professor Dennis Sun (technically just finished). It is a small class, with about twelve people, allowing Professor Sun to dive into the material while maintaining a level of connection with the students. I have greatly appreciated this element as it has allowed me to ask many questions along the way!

In my statistical journey, this class served as a follow-up to STATS118, which I took in the Fall with Professor Gene Kim. Put simply, that class was very difficult. It involved a lot of mathematical computation and required students to pick up patterns and manipulate expressions into known distributions.

As mentioned in the title, STATS200Q feels much more philosophical. It is still no doubt a math class, but it opts for less computation than 118 and more answering of "Why?". Looking back, many of the strategies I learned in 118 were "frequentist". However, even after taking that final exam, I would not have been able to explain what that term meant.

In STATS200Q, we have learned about the differences between frequentist and Bayesian statistics and how they approach problems separately. Each week throughout the quarter, we had a small homework assignment. This final book is an extension of that. Each assignment is its own chapter, and I have additionally added a "Background" section to each chapter. Put together, this should make it feel like a tightly-knit narrative.

This introduction chapter is no different. Sadly, this is the end of the "Background" section. Below I will explain the distribution and dataset I focused my studies on. I hope you enjoy!

NOTE: There are chapters in the book with multiple R blocks. Some R blocks call on variables that were defined in a separate R block earlier in that chapter. So, please make sure to run the R code in order for each chapter.

Also, I couldn't figure out how to get a separate "References" page to work, so when I cite a paper, I do so at the end of the chapter that references the paper.

My Dataset and Distribution

We were each assigned a distribution to focus our analysis on. I chose the Negative Binomial distribution, and the variant that measures how many failures before a successful attempt. I focus my data analysis on a specific COVID hospital dataset, which keeps track of who infected who in that hospital. In my dataset, the data with the negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values).

Source: The original dataset is from this GitHub link: https://github.com/linwangidd/covid19_transmissionPairs_China/blob/master/transmission_pairs_covid_v2.csv.

The PMF for the variant of the Negative Binomial I will be analyzing is:

$$P(X = x) = \binom{x + r - 1}{x} p^r (1 - p)^x$$

If you want to do any analysis of the data, feel free to code in the block below.

`dat` is the dataset.

`x` is the number of people each person infected beyond the first person, and follows a negative binomial distribution.

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
length(x)
```

As seen in the code block above, there are 807 patients in my dataset.

Traditionally, in this version of the Negative Binomial distribution, p is meant to be the probability of success for each trial and r is meant to be the fixed number of successes we are waiting for (which hypothetically would be 1 in this case). However, in the real world, studies have found that negative binomials can effectively model hospital datasets where the parameters have slightly altered meanings (I will mention one such study at the end of this chapter).

In my dataset, p is the probability of a transmission opportunity not resulting in an infection (perhaps counter-intuitively, the lower the p, the more infectious). r is the dispersion parameter, meaning it is a measure of how heavily clustered the transmissions are.

r seems to fit best below 1, meaning there are many individuals who do not pass the virus to anyone beyond the first person. This makes sense as, from the code below, we can see that

507 of the individuals in the dataset have a value of 0, meaning they did not pass the virus to anyone beyond the first person.

```
sum(x == 0)
```

It is important to note that the PMF I wrote out earlier includes r in a choose function. Technically, this is not allowed, as r is not necessarily an integer for my dataset. However, in practice, we can just use the gamma fraction to replace the choose function whenever computations are needed.

For some quick context, there is a great 2005 paper by Lloyd-Smith et al. (2005) titled "Superspreading and the effect of individual variation on disease emergence", which found the negative binomial distribution to be a great fit for a 2002-2004 SARS epidemic in Singapore and Beijing, and their $\hat{r}$ was 0.16.

As a quick note, for this class, we mainly focused on analyzing one parameter. So, for many of these chapters, I will fix r, and analyze p.

1 MLE & Posterior Mean

1.1 Background

This chapter covers the Bayesianand frequentist method for estimating a parameter. Imagine you flip a potentially weighted coin N times, and X are heads. How would you about estimating the true probability that the coin lands on heads? This is an interesting question, and Bayesians and frequentists would take different approaches.

This serves as a motivation for estimating parameters. Below, I will estimate p in the Negative Binomial Distribution.

First, I will first use the Maximum Likelihood Estimation method to estimate p, which is the frequentist approach. Then, I will use a posterior mean, which is the Bayesian approach. Finally, I will compare them using my dataset.

1.2 Maximum Likelihood Estimation

The frequentist method for estimating parameter is finding the MLE. Below I derive the MLE for p in the negative binomial.

$X_1, ..., X_n \sim \text{NegBin}(r, p)$

$$
\begin{aligned}
P(X=x) &= \binom{x+r-1}{x} p^r (1-p)^x \\
L_{x_1,\ldots,x_n}(p) &= \prod_{i=1}^{n} \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i} \\
L_{x_1,\ldots,x_n}(p) &= p^{rn} \prod_{i=1}^{n} \binom{x_i+r-1}{x_i} (1-p)^{x_i} \\
L_{x_1,\ldots,x_n}(p) &= p^{rn} \left(\prod_{i=1}^{n} \binom{x_i+r-1}{x_i} \right) (1-p)^{\sum_{i=1}^{n} x_i} \\
l_{x_1,\ldots,x_n}(p) &= rn \log(p) + \sum_{i=1}^{n} \log\left(\frac{(x_i+r-1)!}{x_i!(r-1)!} \right) + \left(\sum_{i=1}^{n} x_i \right) \log(1-p) \\
\frac{d}{dp} l_{x_1,\ldots,x_n}(p) &= \frac{d}{dp} \left(rn \log(p) + \sum_{i=1}^{n} \log\left(\frac{(x_i+r-1)!}{x_i!(r-1)!} \right) + \left(\sum_{i=1}^{n} x_i \right) \log(1-p) \right) \\
\frac{d}{dp} l_{x_1,\ldots,x_n}(p) &= \left(\frac{rn}{p} - \frac{\sum_{i=1}^{n} x_i}{1-p} \right) \\
0 &= \frac{rn}{p} - \frac{\sum_{i=1}^{n} x_i}{1-p} \\
\frac{rn}{p} &= \frac{\sum_{i=1}^{n} x_i}{1-p} \\
\frac{1-p}{p} &= \frac{\sum_{i=1}^{n} x_i}{rn} \\
\frac{1}{p} - 1 &= \frac{\bar{x}}{r} \\
\hat{p} &= \frac{r}{r+\bar{x}}
\end{aligned}
$$

1.3 Posterior Mean

For the Bayesian method, we need a prior distribution that we will then have the data update. Let's use a Beta(α, β) prior.

$$
\begin{aligned}
f(p|x) &= \frac{f(p) f(x_1,\ldots,x_n|p)}{f(x_1,\ldots,x_n)} \\
f(p|x) &= \frac{\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1} (1-p)^{\beta-1} \prod_{i=1}^{n} \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i}}{f(x_1,\ldots,x_n)} \\
f(p|x) &\propto p^{\alpha+nr-1} (1-p)^{\beta+\sum x_i - 1}
\end{aligned}
$$

This results in $f(p|x) \sim \text{Beta}(\alpha + nr, \beta + \sum_{i=1}^{n} x_i)$.

If we set up our Beta prior with $\alpha = 1, \beta = 1$ (which is actually just a uniform), we get:

$$\mathbb{E}[P] = \frac{\alpha + nr}{\alpha + \beta + nr + \sum_{i=1}^{n} x_i}$$
$$\mathbb{E}[P] = \frac{1 + nr}{2 + nr + \sum_{i=1}^{n} x_i}$$

1.4 My Dataset

Now, let's compute the two estimators on my dataset.

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1

x_bar  <- mean(x)
n      <- length(x)
x_sum  <- sum(x)

cat("x_bar:", x_bar, "\n")
cat("n:", n, "\n")
cat("x_sum:", x_sum, "\n")
```

As we can see from the code above, in my dataset, $\bar{x} = 0.7434944, n = 807, \sum(x) = 600$.

So, plugging this in, we have:

$$\hat{p}_{MLE} = \frac{r}{r + 0.7434944}$$
$$\hat{p}_{posterior} = \frac{1 + 807r}{2 + 807r + 600}$$

If we assume $r = 0.5$:

$$\hat{p}_{MLE} = \frac{0.5}{0.5 + 0.7434944} = 0.402$$
$$\hat{p}_{posterior} = \frac{1 + 807(0.5)}{2 + 807(0.5) + 600} = 0.401$$

1.5 Comparing Estimators

Now, let's compare the bias and variance of the two estimators. For the sake of evaluating our estimators, let's say true p is $\frac{1}{3}$.

In my dataset, there are 807 patients. After 10,000 simulations, the bias and variance of our estimators is:

```
n_obs <- 807
true_size <- 0.5 #r
true_p <- 1 / 3

sims <- replicate(10000, {
  x <- rnbinom(n = n_obs, size = true_size, prob = true_p)
  x_sum <- sum(x)
  x_avg = mean(x)
  p.mle <- true_size / (true_size + x_avg)

  p.pm <- (1 + n_obs * true_size) / (2 + n_obs * true_size + x_sum)

  c(mle = p.mle, pm = p.pm)
})

sims <- t(sims)

bias_mle <- mean(sims[,"mle"]) - true_p
bias_pm <- mean(sims[,"pm"]) - true_p

var_mle <- var(sims[,"mle"])
var_pm <- var(sims[,"pm"])

results <- data.frame(
  Metric = c("Bias", "Variance"),
  MLE = c(bias_mle, var_mle),
  Posterior_Mean = c(bias_pm, var_pm)
)

results
```

As we can see from the table produced by the code, $\hat{p}_{MLE}$ has positive bias while $\hat{p}_{PM}$ is biased toward the prior distribution (which in this case leads to positive bias).

$\hat{p}_{PM}$ has less slightly less variance. This is more apparent if you make `n_obs` smaller in the code.

Here is the the code for a histogram of $\hat{p}_{MLE}$ over many trials.

```
hist_mle <- hist(sims[, "mle"],
     breaks = 30,
     main = expression("Histogram of "* hat(p)[MLE]),
     xlab = expression(hat(p)[MLE])
    )
```

Here is the the code for a histogram of $\hat{p}_{PM}$ over many trials.

```
hist_pm <- hist(sims[, "pm"],
     breaks = 30,
     main = expression("Histogram of "* hat(p)[PM]),
     xlab = expression(hat(p)[PM])
     )
```

2 Further Evaluating Estimators

2.1 Background

This chapter covers a variety of topics. First, I will dive into some cool properties regarding the score, which is the derivative of the log likelihood. One integration technique I will use for this is "Differentiation Under the Integral Sign". It is a cool trick that allows you to move a derivative outside of the integral.

Then, we will take a nice short break from proofs, and I will mention another way to evaluate estimators. In Chapter 1, we looked at the bias and variance of $\hat{p}_{MLE}$ and $\hat{p}_{PM}$, but this time I will use frequentist Mean Squared Error to compare them. This is defined as MSE $= \mathbb{E}[(\hat{\theta}-\theta)^2]$. The MSE can famously be decomposed as the (Bias^2 + Variance). So, while last chapter we looked at bias and variance, MSE serves as an all-encompassing metric. So, I will compare the MSE of my MLE and Posterior Mean from Chapter 1.

Alas, after discussing MSE, our short break from proofs will have concluded. I will next prove the Cramer-Rao Lower Bound (CRLB), evaluate the CRLB for my distribution, and see if it is relevant to the MLE and Posterior Mean I calculated in Chapter 1. In class, we learned that the CRLB is a lower bound to the variance of an unbiased estimator. This means that of all unbiased estimators, if one reaches the CRLB, then it is the best unbiased estimator, as it would have the lowest MSE. This is very insightful. It means once we've reached that bound, there is no point evaluating other unbiased estimators, as there cannot be a unbiased estimator with a lower variance and MSE that the one that reaches the CRLB.

As an extra note, the CRLB is defined as $\frac{1}{I(\theta)}$, where $I(\theta)$ is defined as the variance of the score (which is the derivative of the log-likelihood). I will prove this later on in this chapter. It is important to note that some of the equations I derive in the first part of this chapter will make it easier to solve for the CRLB for my distribution.

2.2 Proving Var$(\ell^{'}(\theta)) = -\mathbb{E}[\ell^{''}(\theta)]$

For any PMF/PDF $f_\theta(x)$, the log-likelihood of x can be written as $\ell_x(\theta)$.

Another name for the derivative of the log-likelihood is the score. The expected value of the score is defined as $\mathbb{E}[\ell'_\theta(x)]$.

$$\mathbb{E}[\ell'_x(\theta)] = \int_{-\infty}^{\infty} \ell'_x(\theta) f_\theta(x) dx$$

Now, let's solve for $\ell'_x(\theta)$:

$$\begin{aligned} L_x(\theta) &= f_\theta(x) \\ \ell_x(\theta) &= \log(f_\theta(x)) \\ \frac{d}{d\theta}\ell_x(\theta) &= \frac{\frac{d}{d\theta} f_\theta(x)}{f_\theta(x)} \\ \ell'_x(\theta) &= \frac{\frac{d}{d\theta} f_\theta(x)}{f_\theta(x)} \end{aligned}$$

Subbing this into our expectation:

$$\begin{aligned} \mathbb{E}[\ell'_x(\theta)] &= \int_{-\infty}^{\infty} \frac{\frac{d}{d\theta} f_\theta(x)}{f_\theta(x)} f_\theta(x) dx \\ &= \int_{-\infty}^{\infty} \frac{d}{d\theta} f_\theta(x) dx \\ &= \frac{d}{d\theta} \int_{-\infty}^{\infty} f_\theta(x) dx \\ &= \frac{d}{d\theta}(1) = 0 \end{aligned}$$

The fact that $\mathbb{E}[\ell'_x(\theta)] = 0$ is an important result and I use it again later on in this chapter.

Now, let's solve for $\text{Var}(\ell'(\theta))$.

We know: $\text{Var}(\ell'(\theta)) = \mathbb{E}[(\ell'(\theta))^2] - (\mathbb{E}[\ell'(\theta)])^2$.

Since $\mathbb{E}[\ell'_x(\theta)] = 0$:

$$\text{Var}(\ell'(\theta)) = \mathbb{E}[(\ell'(\theta))^2] = \int_{-\infty}^{\infty} (\ell'_x(\theta))^2 f_\theta(x) dx$$

Next, we relate $(\ell'_x(\theta))^2$ to the second derivative:

$$\ell''_x(\theta) = \frac{d}{d\theta}\left[\frac{\frac{d}{d\theta}f_\theta(x)}{f_\theta(x)}\right] = \frac{f_\theta(x)\frac{d^2}{d\theta^2}f_\theta(x) - (\frac{d}{d\theta}f_\theta(x))^2}{(f_\theta(x))^2}$$

$$\ell''_x(\theta) = \frac{\frac{d^2}{d\theta^2}f_\theta(x)}{f_\theta(x)} - \left(\frac{\frac{d}{d\theta}f_\theta(x)}{f_\theta(x)}\right)^2$$

$$\ell''_x(\theta) = \frac{\frac{d^2}{d\theta^2}f_\theta(x)}{f_\theta(x)} - (\ell'_x(\theta))^2$$

Rearranging gives $(\ell'_x(\theta))^2 = \frac{\frac{d^2}{d\theta^2}f_\theta(x)}{f_\theta(x)} - \ell''_x(\theta)$.

Subbing this into $\text{Var}(\ell'(\theta)) = \mathbb{E}[(\ell'(\theta))^2] = \int_{-\infty}^{\infty}(\ell'_x(\theta))^2 f_\theta(x)dx$, gives us:

$$\begin{aligned}
\text{Var}(\ell'(\theta)) &= \int_{-\infty}^{\infty} f_\theta(x)\left(\frac{\frac{d^2}{d\theta^2}f_\theta(x)}{f_\theta(x)} - \ell''_x(\theta)\right)dx \\
&= \int_{-\infty}^{\infty} \frac{d^2}{d\theta^2}f_\theta(x) - \ell''_x(\theta)f_\theta(x)dx \\
&= \int_{-\infty}^{\infty} \frac{d^2}{d\theta^2}f_\theta(x)dx - \int_{-\infty}^{\infty}\ell''_x(\theta)f_\theta(x)dx \\
&= \frac{d^2}{d\theta^2}\int_{-\infty}^{\infty} f_\theta(x)dx - \mathbb{E}[\ell''_x(\theta)] \\
&= \frac{d^2}{d\theta^2}(1) - \mathbb{E}[\ell''_x(\theta)] \\
&= -\mathbb{E}[\ell''_x(\theta)]
\end{aligned}$$

So, we have just shown that $\text{Var}(\ell'(\theta)) = -\mathbb{E}[\ell''_x(\theta)]$. This value is also known as the Fisher Information, or $I(\theta)$.

2.3 MSE

In Chapter 1, I derived that if $X_1, ..., X_n \sim \text{NegBin}(r, p)$, then: $\hat{p}_{MLE} = \frac{r}{r+\bar{x}}$ and $\hat{p}_{PM} = \frac{1+nr}{2+nr+\sum x_i}$.

We previously analyzed the bias and variance of these estimators. Now, let's analyze their MSE, which is $\mathbb{E}[(\hat{p}-p)^2]$. As mentioned in the Background section, this can be rewritten as Bias^2 + Variance.

Bbelow is a simulation that prints out Mean Squared Error for different p, for when $n = 50$ and $r = 0.5$.

```
n <- 50
true_size <- 0.5 #r

p_grid <- seq(0.01, 0.99, length.out = 40)
mse_results <- sapply(p_grid, function(p) {

  # Run simulations for each specific p in the grid
  temp_sims <- replicate(10000, {
    x <- rnbinom(n = n, size = true_size, prob = p)
    x_avg = mean(x)
    x_sum = sum(x)

    p.mle <- true_size / (true_size + x_avg)
    p.pm <- (1 + n * true_size) / (2 + n * true_size + x_sum)

    c(mle = p.mle, pm = p.pm)
  })

  # Calculate MSE for this specific p
  mse_mle_p <- mean((temp_sims["mle",] - p)^2)
  mse_pm_p <- mean((temp_sims["pm",] - p)^2)

  c(mse_mle = mse_mle_p, mse_mod = mse_pm_p)
})

# Plotting the curve
plot(p_grid, mse_results["mse_mle",], type = "l", col = "blue", lwd = 2,
     main = "MSE for Different Estimators",
     xlab = "True p",
     ylab = "MSE",
     cex.main = 0.9,
     cex.lab = 0.8,
     cex.axis = 0.7)

lines(p_grid, mse_results["mse_mod",], col = "red", lwd = 2, lty = 2)
legend("topright",
       legend = c("MLE", "Posterior Mean"),
       col = c("blue", "red"),
       lty = 1:2,
       cex = 0.6)
```

As we can see, at different true p values, one estimator may be better than the other. This is

an interesting result, and means that we cannot declare one officially estimator to be better than the other using frequentist MSE.

2.4 Proving the Cramer-Rao Lower Bound

As mentioned in the "Background" section, the CRLB is a lower bound to the variance of an unbiased estimator. Let's show why this is the case.

First, let $t(x)$ be any unbiased estimator of θ. Now let's solve for $\text{Cov}(t(x), \ell'_x(\theta))$.

$$\text{Cov}(t(x), \ell'(x)) = \mathbb{E}[t(x) \cdot \ell'_x(\theta)] - \mathbb{E}[t(x)]\mathbb{E}[\ell'_x(\theta)]$$

Now, in the first section of this Chapter, we showed that $\mathbb{E}[\ell'_x(\theta)] = 0$. So, let's apply that and continue to manipulate.

$$\begin{aligned}\text{Cov}(t(x), \ell'(x)) &= \mathbb{E}[t(x) \cdot \ell'_x(\theta)] - 0 \\ &= \int_{-\infty}^{\infty} t(x)\ell'(x)f_\theta(x)dx\end{aligned}$$

Now, in the first section we also showed that: $\ell'_x(\theta) = \frac{\frac{d}{d\theta}f_\theta(x)}{f_\theta(x)}$

Using this, we can say that:

$$\begin{aligned}\text{Cov}(t(x), \ell'(x)) &= \int_{-\infty}^{\infty} t(x)\frac{\frac{d}{d\theta}f_\theta(x)}{f_\theta(x)}f_\theta(x)dx \\ &= \int_{-\infty}^{\infty} t(x)\frac{d}{d\theta}f_\theta(x)dx \\ &= \frac{d}{d\theta}\int_{-\infty}^{\infty} t(x)f_\theta(x)dx \\ &= \frac{d}{d\theta}\mathbb{E}[t(x)] \\ &= \frac{d}{d\theta}\theta \text{ [NOTE: This is because t(x) is unbiased]} \\ &= 1\end{aligned}$$

So, using the Cauchy-Schwarz Inequality, we can say that

$1 = \text{Cov}^2(t(x), \ell'(x)) \leq \text{Var}(t(x))\text{Var}(\ell'_x(\theta))$

So, this means that:

$\frac{1}{\text{Var}(\ell'_x(\theta))} \leq \text{Var}(t(x))$.

So, this means that the variance of any unbiased estimator $t(x)$ must be greater than or equal to $\frac{1}{\text{Var}(\ell'_x(\theta))}$. And, if you remember earlier, this can be written as:

CRLB: $\frac{1}{\text{Var}(\ell'_x(\theta))} = \frac{1}{I(\theta)} = \frac{1}{-\mathbb{E}[\ell''_x(\theta)]}$

2.5 Finding the CRLB

As described above, the CRLB for unbiased estimators is $\frac{1}{I(\theta)}$, where $I(\theta)$ is defined as the variance of the score. However, we proved above that the variance of the score, $\text{Var}(\ell'(\theta))$ is actually equivalent to $-\mathbb{E}[\ell''(\theta)]$

So, this means that the CRLB for unbiased estimators is $\frac{1}{-\mathbb{E}[\ell''(\theta)]}$.

In Chapter 1, I wrote For $X_1, ..., X_n \sim \text{NegBin}(r, p)$, then $\ell'(p) = \frac{rn}{p} - \frac{\sum_{i=1}^n x_i}{1-p}$:

This means that $\ell''(p) = -\frac{rn}{p^2} - \frac{\sum_{i=1}^n x_i}{(1-p)^2}$

$$\begin{aligned}
\mathbb{E}\left[\ell''(p)\right] &= -\left(\frac{rn}{p^2} + \frac{\mathbb{E}[\sum_{i=1}^n x_i]}{(1-p)^2}\right) \\
&= -\left(\frac{rn}{p^2} + \frac{rn(1-p)}{p(1-p)^2}\right) \\
&= -\frac{rn}{p^2(1-p)}
\end{aligned}$$

So, the lowest possible variance for an unbiased estimator for p is $\frac{p^2(1-p)}{rn}$.

2.6 Does it Apply?

In Week 1, we established from simulation that $\hat{p}_{MLE}$ is positively biased. Additionally, in class we learned that the $\hat{p}_{PM}$ is always biased toward the prior. Since the MLE and Posterior Mean are both biased in this case, the CRLB does not provide a strict lower bound for their variance.

3 Asymptotic Distribution of the MLE

3.1 Background

In Chapter 1, we calculated the $\hat{p}$ of the Negative Binomial distribution. This chapter will dive into the Asymptotic Distribution of the MLE. This is interesting because in general, calculating the bias and variance of the MLE by hand can sometimes be difficult.

This task can be made easier when dealing with large sample sizes by knowing that the distribution of the MLE approximates to a Normal as $n \to \infty$ It is kind of wild to think about no matter the original distribution, the distribution of the MLE always approaches the Normal distribution!

I will first simulate my distribution and look at a histogram of $\hat{p}_{MLE}$ as n grows large. Then, I will prove that the MLE approximates to a Normal as $n \to \infty$ and apply that to my distribution.

3.2 Simulating Distribution

Let's say $X_1, ..., X_n \sim \text{NegBin}(r, p)$, where $r = 0.5$ and $p = \frac{1}{3}$.

We derived in Chapter 1. that $\hat{p}_{MLE} = \frac{r}{r+\bar{x}}$. Below is a histogram of $\hat{p}_{MLE}$ as n grows large, in this case to $10,000$.

```
n_obs <- 10000
true_size <- 0.5 #r
true_p <- 1 / 3

sims <- replicate(1000, {
  x <- rnbinom(n = n_obs, size = true_size, prob = true_p)
  x_sum <- sum(x)
  x_avg = mean(x)
  p.mle <- true_size / (true_size + x_avg)

  p.mle
})
```

```
sims <- t(sims)

hist_mle <- hist(sims,
     breaks = 100,
     main = expression("Histogram of "* hat(p)[MLE]),
     xlab = expression(hat(p)[MLE])
)
```

As we can see, this looks pretty Normal, and it looks to be centered around $\frac{1}{3}$, which is what we set true p to be.

3.3 Proof of MLE Approaching a Normal Distribution

Now let's prove that the distribution of the MLE approaches a Normal distribution as n gets large. This proof is a really cool one and involves many different concepts.

Let's start with the score of $\hat{\theta}_{MLE}$, which is $\ell'(\hat{\theta}_{MLE})$

Let's now Taylor expand $\ell'(\hat{\theta}_{MLE})$ around θ_0.

$\ell'(\hat{\theta}_{MLE}) \approx \ell'(\theta_0) + \ell''(\theta_0)(\hat{\theta}_{MLE} - \theta_0)$

Now, remember that when solving for the MLE (as we did in Chapter 1), we set $\ell'(x) = 0$. This is because the MLE is defined as the θ value that maximizes the likelihood (and log-likelihood).

So, as the MLE is a maximum of the log-likelihood function, this means that $\ell'(\hat{\theta}_{MLE}) = 0$.

So, plugging this in we get:

$0 = \ell'(\hat{\theta}_{MLE}) \approx \ell'(\theta_0) + \ell''(\theta_0)(\hat{\theta}_{MLE} - \theta_0)$

We can rearrange this to say:

$$\begin{aligned} 0 &\approx \ell'(\theta_0) + \ell''(\theta_0)(\hat{\theta}_{MLE} - \theta_0) \\ -\ell'(\theta_0) &\approx \ell''(\theta_0)(\hat{\theta}_{MLE} - \theta_0) \\ \hat{\theta}_{MLE} - \theta_0 &\approx \frac{-\ell'(\theta_0)}{\ell''(\theta_0)} \end{aligned}$$

Now, let's do some further manipulation.

$$\hat{\theta}_{MLE} - \theta_0 \approx \frac{-\ell'(\theta_0)}{\ell''(\theta_0)}$$
$$\hat{\theta}_{MLE} - \theta_0 = \frac{\frac{1}{n}\ell'(\theta_0)}{-\frac{1}{n}\ell''(\theta_0)}$$
$$\hat{\theta}_{MLE} - \theta_0 = \frac{\frac{1}{n}\sum_{i=1}^{n}\ell'_{x_i}(\theta_0)}{-\frac{1}{n}\sum_{i=1}^{n}\ell''_{x_i}(\theta_0)}$$
$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) = \frac{\sqrt{n}(\frac{1}{n}\sum_{i=1}^{n}\ell'_{x_i}(\theta_0))}{-\frac{1}{n}\sum_{i=1}^{n}\ell''_{x_i}(\theta_0)}$$

Now, let's say $Y = \ell'_{x_1}(\theta_0)$. This means that $\bar{Y} = \frac{1}{n}\sum_{i=1}^{n}\ell'_{x_i}(\theta_0)$.
This means that:

$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) = \frac{\sqrt{n}(\bar{Y} - 0)}{-\frac{1}{n}\sum_{i=1}^{n}\ell''_{x_i}(\theta_0)}$$

Now, we can realize three things.
First, the CLT is defined as $\sqrt{n}(\bar{Y} - \mathbb{E}[Y_1]) \to N(0, Var(Y_1))$
Second, as we showed in Chapter 2, $\mathbb{E}[Y_1] = \mathbb{E}[\ell'_{x_1}(\theta_0)] = 0$.
Third, also from Chapter 2, we showed that $I(\theta) = -\mathbb{E}[\ell''(\theta)] = \text{Var}(\ell'(\theta))$
So, this means that:

$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \to \frac{N(0, \text{Var}(\ell'_{x_1}(\theta_0))}{\text{Var}(\ell'(\theta_0))}$$
$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \to N\Big(0, \frac{1}{\text{Var}(\ell'_{x_1}(\theta_0))}\Big)$$
$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \to N\Big(0, \frac{1}{I_{x_1}(\theta)}\Big)$$

Isolating for $\hat{p}_{MLE}$, we have that $\hat{p}_{MLE} \to N\left(p, \frac{1}{nI_{x_1}(\theta)}\right)$

3.4 Asymptotic Calculation

As shown above, using the CLT and Taylor approximation, we can see that as n grows large, we have that:

$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \to N\Big(0, \frac{1}{I_{x_1}(\theta)}\Big)$

So, let's solve for $I_{x_1}(\theta)$. Once again, from Chapter 2, we know that $I(\theta) = -\mathbb{E}[\ell''(\theta)] = \text{Var}(\ell'(\theta))$

Taking the score that we got from Chapter 1 (while solving for the MLE), we have:

$$\frac{d}{dp}\ell_{x_1,\ldots,x_n}(p) = \left(\frac{rn}{p} - \frac{\sum x}{1-p}\right)$$

As we only want to work with one observation now, we will set $n = 1$.

$$\ell'_{x_1}(p) = \left(\frac{r}{p} - \frac{x}{1-p}\right)$$

Let's solve for $-\mathbb{E}[\ell''(\theta)]$.

$$\begin{aligned}
\ell'_{x_1}(p) &= \frac{r}{p} - \frac{x}{1-p} \\
\frac{d^2}{dp^2}\ell_{x_1}(p) &= \frac{-r}{p^2} - \frac{x}{(1-p)^2} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2} + \frac{\mathbb{E}[x]}{(1-p)^2} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2} + \frac{\left(\frac{r(1-p)}{p}\right)}{(1-p)^2} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2} + \frac{r}{p(1-p)} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r(1-p)}{p^2(1-p)} + \frac{rp}{p^2(1-p)} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r(1-p)+rp}{p^2(1-p)} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r(1-p+p)}{p^2(1-p)} \\
-\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2(1-p)}
\end{aligned}$$

So, we have

$$\sqrt{n}(\hat{p}_{MLE} - p) \to N\left(0, \frac{p^2(1-p)}{r}\right)$$

And again isolating for $\hat{p}_{MLE}$, we have:

$$\hat{p}_{MLE} \dot{\sim} N\left(p, \frac{p^2(1-p)}{nr}\right)$$

3.5 Graphing Asymptotic MLE Curve

Below is a curve with the distribution of $\hat{p}_{MLE}$ with $r = 0.5$, $p = \frac{1}{3}$, and large n, let's go with 10,000, behind a red curve displaying the asymptotically normal distribution of $\hat{p}_{MLE}$.

```
hist_mle <- hist(sims,
     breaks = 100,
     main = expression("Histogram of "* hat(p)[MLE]),
     xlab = expression(hat(p)[MLE]),
     freq=F
)
curve(dnorm(x, mean=true_p,
            sd= sqrt( ( (true_p ** 2) * (1 - true_p) ) / (n_obs * true_size))),
add=T, col="red", lwd=2)

legend("topright", legend = expression("Asymptotic Normal "* hat(p)[MLE]),
       col="red", lwd=2, lty = 1, cex = 0.6)
```

4 Confidence and Credible Intervals

4.1 Background

So far, we have discussed solving for and evaluating $\hat{p}_{MLE}$ and $\hat{p}_{PM}$. But, these estimators are based only on the data we have, and neither is likely to be the true p of the entire population.

So, this chapter is all about Confidence and Credible Intervals. The former is a frequentist method to display a region that they believe contains the true p of the entire population with a certain confidence. The latter is the Bayesian version that allows for slightly different wording that I will explain in the next paragraph. For these examples, we use a 95% confidence and credible interval.

This is the topic that truly instilled in me the differences in frequentist v. Bayesian principles. As frequentists treat p as a fixed but unknown parameter, once they have calculated their region, they do NOT say that there is a 95% chance that p is in the region. This is because p is fixed. A fixed parameter cannot have a chance of being in a region, it either is or it isn't. So, they escape this by saying they are 95% "confident" that p is contained in the region. On the the other hand, as Bayesians treat p is a random variable, they have no problems saying that there is a 95% chance that p is in their credible interval.

4.2 Confidence Interval

To remember, in Chapter 1, we solved that for $X_1, ..., X_n \sim \text{NegBin}(r, p)$ $\hat{p}_{MLE} = \frac{r}{r+\bar{x}}$.

In Chapter 3, we established both that

$$\sqrt{n}(\hat{p}_{MLE} - p) \sim N\left(0, \frac{p^2(1-p)}{r}\right)$$

and

$$\hat{p}_{MLE} \dot{\sim} N\left(p, \frac{p^2(1-p)}{nr}\right)$$

This means that as the number of data points approaches infinity, $\hat{p}_{MLE}$ is approximately Normal, with a mean of p and a variance of $\frac{p^2(1-p)}{nr}$.

Therefore, an approximate 95% confidence interval for p is: $\hat{p}_{MLE} \pm 2\sqrt{\frac{p^2(1-p)}{nr}}$. However, we do not know the true value of p, so we have two options in how we can proceed.

4.2.1 Plug-In

We can try plugging in $\hat{p}_{MLE}$ for p in this equation. This means that the approximate 95% confidence interval is:

$$\begin{aligned}
\hat{p} \pm 2\sqrt{\frac{\hat{p}^2_{MLE}(1-\hat{p}_{MLE})}{nr}} &= \hat{p} \pm 2\sqrt{\frac{\left(\frac{r}{r+\bar{x}}\right)^2\left(1-\frac{r}{r+\bar{x}}\right)}{nr}} \\
&= \hat{p} \pm 2\sqrt{\frac{\left(\frac{r}{r+\bar{x}}\right)^2\left(\frac{r+\bar{x}}{r+\bar{x}}-\frac{r}{r+\bar{x}}\right)}{nr}} \\
&= \hat{p} \pm 2\sqrt{\frac{\left(\frac{r}{r+\bar{x}}\right)^2\left(\frac{\bar{x}}{r+\bar{x}}\right)}{nr}} \\
&= \hat{p} \pm 2\sqrt{\frac{r^2\bar{x}}{nr(r+\bar{x})^3}} \\
&= \hat{p} \pm 2\sqrt{\frac{r\bar{x}}{n(r+\bar{x})^3}}
\end{aligned}$$

Going back to my hospital dataset:

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
x_bar = mean(x)
n = length(x)
cat("x_bar =", x_bar, "\tn =", n)
```

As we can see, I had $\bar{x} \approx 0.7435, n = 807$. Let's say $r = 0.5$.

I then have:

$$\hat{p} \pm 2\sqrt{\frac{r\bar{x}}{n(r+\bar{x})^3}}$$

$$\frac{r}{r+\bar{x}} \pm 2\sqrt{\frac{r\bar{x}}{n(r+\bar{x})^3}}$$

$$\frac{0.5}{0.5+0.7435} \pm 2\sqrt{\frac{(0.5)(0.7435)}{807(0.5+0.7435)^3}}$$

$$0.4021 \pm 2\sqrt{0.00023958}$$

So, the approximate plug-in 95% confidence interval for p is $[0.3711, 0.4331]$.

4.2.2 Direct Computation

Instead of plugging in $\hat{p}_{MLE}$ for p, we can try to solve for what values of p make up the confidence interval.

We established earlier that:

$\sqrt{n}(\hat{p}_{MLE} - p) \sim N\left(0, \frac{p^2(1-p)}{r}\right)$

This means that: $\hat{p}_{MLE} - p \sim N\left(0, \frac{p^2(1-p)}{nr}\right)$.

Furthermore, we can say that:

$\frac{\hat{p}_{MLE}-p}{\sqrt{\frac{p^2(1-p)}{nr}}} \sim N(0,1)$

In the Normal distribution, 95% of the mass is in the mean ± 1.96 Standard deviations. As the Normal above has a standard deviation of 1, we can set up a 95% confidence interval that rounds to:

$-2 \leq \frac{\hat{p}_{MLE}-p}{\sqrt{\frac{p^2(1-p)}{nr}}} \leq 2$

$$\left(\frac{\hat{p}_{MLE}-p}{\sqrt{\frac{p^2(1-p)}{nr}}}\right)^2 \leq 4$$
$$\frac{(\hat{p}_{MLE}-p)^2}{\frac{p^2(1-p)}{nr}} \leq 4$$
$$\frac{nr(\hat{p}_{MLE}-p)^2}{p^2(1-p)} \leq 4$$
$$\hat{p}_{MLE}^2 - 2\hat{p}_{MLE}p + p^2 \leq \frac{4}{nr}p^2(1-p)$$
$$\hat{p}_{MLE}^2 - 2\hat{p}_{MLE}p + p^2 - \frac{4p^2}{nr} + \frac{4p^3}{nr} \leq 0$$
$$\frac{4}{nr}p^3 + (1-\frac{4}{nr})p^2 - 2\hat{p}_{MLE}p + \hat{p}_{MLE}^2 \leq 0$$

This is unfortunately not cleanly solvable for p, but we can graph it. If we say that $f(p) = \frac{4}{nr}p^3 + (1-\frac{4}{nr})p^2 - 2\hat{p}_{MLE}p + \hat{p}_{MLE}^2$, we can look at the zeros to see the values of p at the edge of the confidence intervals.

For this sample, let's go back to the dataset I started with, where $\bar{x} = 0.7435, n = 807$. Let's fix $r = 0.5$.

```
r <- 0.5

p_hat <- r / (r + x_bar)

# f(p)
cubic_f <- function(p) {
  a <- 4 / (n * r)
  (a * p^3) + (1 - a) * p^2 - (2 * p_hat) * p + p_hat^2
}

# roots of f(p) the windows [0, p_hat] and [p_hat, 1]
lower_bound <- uniroot(cubic_f, interval = c(0, p_hat))$root
upper_bound <- uniroot(cubic_f, interval = c(p_hat, 1))$root


cat("95% Confidence Interval: [", round(lower_bound, 4), ",", round(upper_bound, 4), "]\n")
```

From the computation in the code above, we are approximately 95% confident that p is in $[0.3727, 0.4346]$

4.3 Credible Interval

In Chapter 1, I solved that given a Beta(α, β) prior, the posterior distribution for p is:

$f(p|x_1, .., x_n) \sim \text{Beta}(\alpha + nr, \beta + \sum_{i=1}^{n} x_i)$.

In Chapter 1, I used a uniform prior, meaning $\alpha = 1, \beta = 1$. This means the posterior distribution given a uniform prior is:

$f(p|x_1, .., x_n) \sim \text{Beta}(1 + nr, 1 + \sum_{i=1}^{n} x_i)$.

For our 95% credible interval, we want the interval to cover 95% of the distribution with symmetric tails.

So, we need a left bound (LB) and right bound (RB), such that:

$\int_0^{LB} \frac{p^{(1+nr)-1}(1-p)^{(1+\sum x_i)-1}}{B(1+nr, 1+\sum x_i)} dp = 0.025$

$\int_{RB}^1 \frac{p^{(1+nr)-1}(1-p)^{(1+\sum x_i)-1}}{B(1+nr, 1+\sum x_i)} dp = 0.025$

Unfortunately, we can't solve for LB and RB by hand. However, the computer can solve it for us, using the `qbeta` function.

Again, we will use the same values from my hospital dataset, as before.

```
r <- 0.5
sum_xi <- x_bar * n
credible_range <- 0.95

# Posterior Parameters based on f(p | x) ~ Beta(1 + nr, 1 + sum x_i)
alpha_post <- 1 + (n * r)
beta_post  <- 1 + sum_xi

# Calculate Tail Areas
alpha_level <- 1 - credible_range
tail_area   <- alpha_level / 2

# Solve for Credible Interval Bounds
LB <- qbeta(tail_area,
            shape1 = alpha_post,
            shape2 = beta_post,
            lower.tail = TRUE)

RB <- qbeta(tail_area,
            shape1 = alpha_post,
            shape2 = beta_post,
```

```
            lower.tail = FALSE)

credible_interval <- c(LB, RB)
names(credible_interval) <- c("Left Bound", "Right Bound")

print(credible_interval)
```

From the computation in the code above, conditional on my uniform prior, the credible interval states that there is a 95% chance p is in $[0.3722, 0.4328]$.

5 Simulating Confidence and Credible Intervals

5.1 Background

In the previous chapter, we talked about using confidence and credible intervals. For the frequentist calculation, we assumed large n and used a Normal approximation for $\hat{p}_{MLE}$. For our credible interval, we chose a conjugate prior so we could solve for the posterior distribution.

However, what if we didn't want to resort to using a Normal approximation for our confidence interval? Additionally, what if we don't know the our posterior distribution for the credible interval?

Statisticians have had come up with computationally-intense mechanisms to work around these issues. For frequentists, this involves the parametric and nonparametric bootstrap. In the former, you assume your population follows a certain distribution, then use the MLE from your sample to generate more datasets. In the latter, you re-sample more datasets from your original sample with replacement. For both, you then look at the MLEs of your many generated datasets.

The Bayesian Process involves a Metropolis-Hastings Markov Chain Monte Carlo. Let's say we want to come up with estimator for θ in Distribution B. To do this, we first pick a starting Distribution A (that has the same domain as Distribution B) and continuously sample from it. After each sample, we establish a probability based on the likelihood that the sample could have been generated by Distributions A & B. We then keep the sample with this probability. The idea, is after a burn-in period, despite being generated from Distribution A, the kept samples will actually resemble Distribution B. I know this can be confusing, and I will explain the intricacies in more detail later in the chapter.

5.2 Bootstrapping

First, let's bring my hospital dataset into R.

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
```

5.2.1 Parametric Bootstrap

If we say $X \sim \text{NegBin}(r, p)$, and we have $r = 0.5$, then as I calculated in Chapter 1, $\hat{p}_{MLE} = \frac{r}{r+\bar{x}} = \frac{0.5}{0.5+0.7435} = 0.402$.

```
r = 0.5
n = length(x)
x_bar = sum(x) / n
p_mle = r / (r + x_bar)
```

So, using a parametric bootstrap, we can simulate many new datasets from the Negabtive Binomial with my $\hat{p}_{MLE}$. Then, we can calculate the MLE of each of these datasets. A histogram of the MLE values of these datasets can be seen in the code block below.

```
p.pboot <- replicate(10000, {
    # Generate dataset
    x.pboot = rnbinom(n, r, p_mle)

    # Calculate new MLE
    x.pboot.bar = sum(x.pboot) / n
    p.pboot.mle = r / (r + x.pboot.bar)
    p.pboot.mle
  }
)

hist(p.pboot)
```

A 95% confidence interval is generated in the code block below.

```
quantile(p.pboot, c(.025, .975))
```

So, our approximate 95% confidence interval for p using Parametric Bootstrapping is about $[0.3738, 0.4336]$, depending on the simulation.

5.2.2 Nonparametric Bootstrap

for the nonparametric boostrap, we just re-sample new datasets from our sample with replacement, and take the MLE sfrom each of these simulated datasets. A histogram of the MLE values of these datasets can be seen in the code block below.

```
p.npboot <- replicate(10000, {

    # Generate new sample (with replacement) from our above sample
    x.npboot <- sample(x, size=n, replace=TRUE)

    # Calculate new MLE
    x.npboot.bar = sum(x.npboot) / n
    p.npboot.mle = r / (r + x.npboot.bar)
    p.npboot.mle
  }
)

hist(p.npboot)
```

A 95% confidence interval is generated in the code block below.

```
quantile(p.npboot, c(.025, .975))
```

So, our approximate 95% confidence interval for p using Nonparametric Bootstrapping is about $[0.3714, 0.4360]$, depending on the simulation.

5.2.3 Comparing Confidence Intervals

Combining last week and this week, I have now computed four different 95% confidence intervals for p.

They are listed below:

1. Using CLT with $\hat{p}_{MLE}$ plug in for p in $I(p)$: $[0.3711, 0.4331]$
2. Using CLT with direct computation for p in $I(p)$: $[0.3727, 0.4346]$
3. Parametric Bootstrap: $[0.3738, 0.4336]$ (with volatility depending on simulation)
4. Nonparametric Bootstrap: $[0.3714, 0.4360]$ (with volatility depending on simulation)

Computing for the range of the 95% confidence interval, we have:

Using CLT with $\hat{p}_{MLE}$ plug in for p in $I(p)$: 0.0620

Using CLT with direct computation for p in $I(p)$: 0.0619

Parametric Bootstrap: 0.0598

Nonparametric Bootstrap: 0.0646

So, using the first simulations I ran, the parametric bootstrap's 95% confidence interval had the smallest range, while the nonparametric bootstrap had the largest range. I am not sure if this means anything, as this may have been strictly due to a weird simulation.

5.3 Metropolis Hastings

In Chapter 1, I calculated a Bayesian posterior mean for my data using a Beta prior. I solved that:

$f(p|x) \propto p^{\alpha+nr-1}(1-p)^{\beta+\sum x_i-1}$.

For my Beta$(1,1)$ (Uniform) prior, this means that:

$f(p|x) \propto p^{1+nr-1}(1-p)^{1+\sum x_i-1}$

$f(p|x) \propto p^{nr}(1-p)^{\sum x_i}$.

Now, instead of recognizing the beta distribution, let's instead run the Metropolis Hastings Algorithm.

The algorithm will work like this. Imagine we start with a list of length one that contains one random sample from my prior distribution (uniform in my case). I will then generate another sample from my prior distribution and compare it to the last element in my list (which in this case would just be the first generated sample).

With a certain likelihood (that I will explain in a second), I will add this new generated sample to my list. With probability [1 − that_likelihood], I will forget about this new sample and just add that last value in my list, to my list again. I will then generate more and more samples and continue this process over and over.

To write out the probability from earlier, imagine f is the prior distribution, g is the distribution we are interested in, and we are comparing new sample x_j and previous sample x_{j-1}. The likelihood of adding generated sample x_j to the list is $\frac{g(x_j)}{g(x_{j-1})} \cdot \frac{f(x_{j-1})}{f(x_j)}$. The likelihood of adding x_{j-1} again is $1 - \left(\frac{g(x_j)}{g(x_{j-1})} \cdot \frac{f(x_{j-1})}{f(x_j)}\right)$

However, as I am using a Uniform$(0,1)$ prior, $f(x) = 1$ for every x. This means that the we keep the generated sample with likelihood $\frac{g(x_j)}{g(x_{j-1})}$, and forget the generated sample and add the older sample to our list again with likelihood $\left(1 - \frac{g(x_j)}{g(x_{j-1})}\right)$.

After generating enough samples, and then removing the first chunk of values from my list (called a burn-in period), the remainder of my list should resemble g, the posterior distribution, which is the Negative Binomial in my case.

To avoid numerical underflow, we will compute $e^{log(f(p|x))}$.

$f(p|x) \propto e^{nr\log(p)+\sum x_i \log(1-p)+C}$.

I will generate 15000 samples and have a burn-in period of 5000.

```
# This function calculates the posterior probability (without accounting for the constant ter
# To avoid numerical underflow, we will first calculate the log, and then do e ^ (log(prob))

post <- function(p) {
  # calculate log posterior (for numerical stability)
  log.p.post <- log(p) * (n * r) +  sum(x) * log(1-p)
  exp(log.p.post)
}

# Start with an arbitrary value of p.
p.post <- c(0.5)

# Run Metropolis-Hastings for 15000 steps
for(i in 1:15000) {

  p.curr <- p.post[length(p.post)]

  # Propose a new value of p. Let's choose from uniform(0,1). Let's call this q.
  p.prop <- runif(1)

  # calculate probability of choosing new_p.
  # This is (post(new) * q(old)) / (post(old) ( q(new)))
  prob <- min(1,
    (post(p.prop) * dunif(p.curr, 0, 1) /
      (post(p.curr) * dunif(p.prop, 0, 1)))
  )

  # add new p value to list
  p.new <- sample(c(p.prop, p.curr),
                  size=1, prob=c(prob, 1-prob))
  p.post <- c(p.post, p.new)
}

# Keep the latter 10,000 samples
late.p.post <- p.post[5001:15000]

# We can estimate the posterior mean by taking the mean of these samples.
mean(late.p.post)
```

Here, from running the code above, my Metropolis-Hastings posterior mean is around 0.401, depending on the simulation.

In Chapter 1, instead of using a simulation, I recognized that my posterior followed a Beta$(1 + nr, 1 + \sum_{i=1}^{n} x_i)$ distribution. In that situation, my posterior mean was 0.401.

Now, let's look at the distribution of my Metropolis-Hastings $\hat{p}$ in the code block below.

```
hist(late.p.post)
```

Also, let's generate a credible interval from my Metropolis-Hastings data from the code block below.

```
quantile(late.p.post, c(.025, .975))
```

Here, my Metropolis-Hastings approximate 95% credible interval is around $[0.3701, 0.4283]$, depending on the simulation.

When I did this from the Beta function in Chapter 4 using the `qbeta` function, my 95% credible interval was $[0.3722, 0.4328]$.

So let's further compare about these 95% credible intervals:

Range:

Direct computation of the posterior distribution: 0.0606

Metropolis-Hastings: 0.0582

So, my approximate M-H 95% credible interval had a slightly smaller spread, though again, this may have just been because of a weird simulation.

6 Hypothesis Testing

6.1 Background

We have previously covered solving for $\hat{p}_{MLE}$ and $\hat{p}_{PM}$, evaluating them, and setting up confidence and credible intervals. Now, we will talk about a new topic: hypothesis testing.

This chapter is the first of three about hypothesis testing. The idea is that we have a default hypothesis about a parameter θ. Let's say, for example, our null hypothesis is that $\theta_0 = 10$. Maybe this represents the number of symptoms per day of medical patients.

Now, let's say we are trying out a new drug and want to see if it is effective. So, maybe our alternative hypothesis is that $\theta_A = 5$, thus lowering the number of symptoms per day. Hypothesis testing is seeing whether, based on the data, we can reject the null hypothesis and accept the alternative.

Unlike when we solved for $\hat{p}_{MLE}$ and $\hat{p}_{PM}$, the result of a hypothesis test is strictly binary. It is only where we accept/reject the null hypothesis.

Frequentist hypothesis testing is all about the Neyman-Person lemma. This says that if we have a distribution $f_\theta(x)$, and $H_0 : \theta = \theta_0$ and $H_A : \theta = \theta_A$, then we want to reject H_0 when $\frac{f_{\theta_A}(x)}{f_{\theta_0}(x)}$ is maximized. This is in order to maximize the power of our hypothesis test.

To understand why this works, let's look at hypothesis testing as a constrained optimization problem. Theoretically we would want our test to never make a mistake. This means we would want it to only accept the null when H_0 is true and only reject when H_0 is false. In reality, there is a strict trade-off between committing a Type I error (rejecting H_0 despite H_0 being true) and maximizing power (correctly rejecting H_0). We handle this by setting a hard cap on our Type I error (which is referred to as α), typically at $\alpha = 0.05$. Type I error can be very harmful, for example leading us to deem a drug as effective even if it isn't. This is why we cap it at 0.05.

Once this budget is locked in, the Neyman-Pearson lemma tells us exactly we can maximize our power (correctly rejecting H_0) while staying within our α allocation. You can think of the likelihood ratio $\frac{f_{\theta_A}(x)}{f_{\theta_0}(x)}$ as a benefit-to-cost efficiency metric for every possible data outcome. The numerator represents the "benefit" (gaining power if H_A is true), while the denominator represents the "cost" (eating away at our fixed α budget if H_0 is true). By sorting our outcomes and choosing to reject H_0 only when this ratio is at its highest, the lemma mathematically

guarantees that we have constructed the most powerful test possible for our chosen significance level.

In class, we showed that this principle extends into one-sided hypothesis testing. Imagine instead of saying, $H_A : \theta = 5$, we said $H_A : \theta < 10$. To solve for this, we want to find a ratio that maximizes the power for every possible θ below 10. What we showed in class is that we can just choose any number below 10 (let's say 2 for this example), and the values the maximize $\frac{f_{\theta_A=2}(x)}{f_{\theta_0}(x)}$ are the same values that maximize $\frac{f_{\theta_A=7}(x)}{f_{\theta_0}(x)}$ or $\frac{f_{\theta_A=9}(x)}{f_{\theta_0}(x)}$, or any value below 10. So, we can just choose one one these values below 10 to set up the ratio for $H_A : \theta < 10$.

Because this single ratio maximizes the power across the entire range of possible one-sided alternative values simultaneously, the resulting test is called a Uniformly Most Powerful (UMP) test.

6.2 Uniformly Most Powerful Test

Let's apply this to my dataset. First, let's bring my hospital dataset into R.

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
```

In my dataset, remember that the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values). Google says that for the Flu, the average person with the disease spreads it to 1-2 people. As our negative binomial distribution only counts the number of additional people each person infects after the first, then setting $E[X] = 0.5$ feels fair. In other words, for the flu, it reasonable to stay the average person in a hospital with a disease infects 0.5 extra people. That would means that $\mathbb{E}[X] = \frac{r(1-p)}{p} = 0.5$.

Isolating for p, we get:

$$
\begin{aligned}
\mathbb{E}[X] &= \frac{r(1-p)}{p} \\
\frac{p}{1-p} &= \frac{r}{\mathbb{E}[X]} \\
\frac{1-p}{p} &= \frac{\mathbb{E}[X]}{r} \\
\frac{1}{p} - 1 &= \frac{\mathbb{E}[X]}{r} \\
\frac{1}{p} &= \frac{\mathbb{E}[X] + r}{r} \\
p &= \frac{r}{\mathbb{E}[X] + r}
\end{aligned}
$$

So, we set $\mathbb{E}[x]$ to be 0.5. Let's say for the sake of this example that we set r to be 0.75. In this scenario, we get $p = \frac{0.75}{0.5+0.75} = 0.6$.

So, our null hypothesis is H_0: $p = 0.6$. Now, we are curious if maybe COVID is infectious than the flu (not considering other considerations like how my dataset is limited to people only already at the hospital). This would mean that $E[X]$ is greater than 0.5, and as $\mathbb{E}[X] = \frac{r(1-p)}{p}$, a smaller p leads to a larger $E[X]$.

This means that:

H_0: $p = 0.6$

H_A: $p < 0.6$

As mentioned in the Background section, to figure out the best X values to reject when $p < 0.6$, all we need to do is choose one $p < 0.6$, and the same will hold true for rest. Let's proceed with $p = 0.3$.

Again, let's proceed with $r = 0.75$.

NegBin(r, p) follows the distribution $\binom{x+r-1}{x}p^r(1-p)^x$:

So, our H_0 gives us NegBin$(r, p = 0.6)$, which follows the distribution: $f_0(x) = \prod_{i=1}^{n} \binom{x_i+r-1}{x_i}0.6^r(1-0.6)^{x_i}$.

Our H_A gives us NegBin$(r, p = 0.3)$, which follows the distribution: $f_A(x) = \prod_{i=1}^{n} \binom{x_i+r-1}{x_i}0.3^r(1-0.3)^{x_i}$.

We want to to maximize $\frac{f_A(x)}{f_0(x)} = \frac{\prod_{i=1}^{n}\binom{x_i+r-1}{x_i}0.3^r(1-0.3)^{x_i}}{\prod_{i=1}^{n}\binom{x_i+r-1}{x_i}0.6^r(1-0.6)^{x_i}}$, as these are the X values that we maximize our $\frac{\text{Power}}{\alpha}$. In other words, when this value ratio is maximized, we have the best ratio for decreasing the odds of a Type II Error while still protecting against Type I Errors.

So, let's maximize:

$$
\begin{aligned}
\frac{f_A(x)}{f_0(x)} &= \frac{\prod_{i=1}^n \binom{x_i+r-1}{x} 0.3^r (1-0.3)^{x_i}}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} 0.6^r (1-0.6)^{x_i}} \\
\frac{f_A(x)}{f_0(x)} &= \frac{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i}\right) 0.3^{nr} (1-0.3)^{\sum x_i}}{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i}\right) 0.6^{nr} (1-0.6)^{\sum x_i}} \\
&= \frac{0.3^{nr}(1-0.3)^{\sum_{i=1}^n x_i}}{0.6^{nr}(1-0.6)^{\sum_{i=1}^n x_i}} \\
&\propto \frac{0.7^{\sum_{i=1}^n x_i}}{0.4^{\sum_{i=1}^n x_i}} \\
&= \left(\frac{0.7}{0.4}\right)^{\sum_{i=1}^n x_i}
\end{aligned}
$$

To maximize, $\left(\frac{0.7}{0.4}\right)^{\sum_{i=1}^n x_i}$, we want to reject when $\sum_{i=1}^n x_i$ is as large as possible.

So, we this means we set up: $P(\sum_{i=1}^n x_i \geq c) = 0.05$.

The sum of many NegBin(p, r) random variables is NegBin$(n \cdot r, p)$, My COVID hospital dataset had 807 patients, so $n = 807$.

```
n = length(x)
n
```

So, we would use the `qnbinom` R function to set up a sum from c to ∞ over the H_0 PMF that covers the upper 5% (at most) of the distribution.

```
r = 0.75
alpha = 0.05
p = 0.6
reject_val = qnbinom(1 - alpha, r * n, p)
reject_val
```

We then check to see how much of the mass is in this value or above (I write `reject_val - 1` because we want the mass above whichever value is inputted into the `pnbinom` function):

```
1 - pnbinom(reject_val - 1, n * r, p)
```

We are above 0.05. So, we need to move our rejection bound up one value, to take up less of the mass.

```
new_reject_val = reject_val + 1
1 - pnbinom(new_reject_val - 1, n * r, p)
```

α is now below 0.05!

```
new_reject_val
```

So, $c = 448$

So, we reject if H_0 when $\sum x_i \geq 448$.

Now, let's see if reject H_0 in my dataset.

```
observed_sum <- sum(x)
observed_sum
```

As $600 > c$ (which was 448) in my COVID hospital dataset, we reject H_0. This implies that we accept the alternative hypothesis that COVID is more contagious than the flu (though this dataset is not perfect as it only consists of people already in the hospital who infected at least 1 person, so I do not want to extrapolate too much).

7 Generalized Likelihood Ratio Test

7.1 Background

Last chapter, we talked about hypothesis testing and the Uniformly Most Powerful Test. Tests like these are possible when our H_A is a greater than or less than, like $H_A : \theta < 1$. However, what if we had $H_A : \theta \neq 1$.

What maximizes power for $\theta = 0.5$ is different than what maximizes power for $\theta = 1.5$. So, we cannot have a Uniformly Most Powerful Test. Instead, we set up the Generalized Likelihood Ratio Test. The idea is still the same, where we still want to find the x values that maximize $\frac{f_{\text{Alternative Hypothesis}}(x)}{f_{\text{Null Hypothesis}}(x)}$. The problem is that if $H_A : \theta \neq 1$, different values maximize $f_{\text{Alternative Hypothesis}}(x)$ depending on if θ is above or below 1.

So, our solution is to find a θ that maximizes the numerator. This means that we can re-write the ratio we want to maximize as: $\frac{\max_\theta \mathcal{L}(\theta)}{\mathcal{L}(\theta_0)}$.

The θ that maximizes the likelihood is the MLE. This means that we want to maximize the ratio $\frac{f_{\hat{\theta}_{MLE}}(x)}{f_{\theta_0}(x)}$.

7.2 Generalized Likelihood Ratio Test

So, let's apply this to my dataset. First, let's bring my hospital dataset into R.

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
```

I mention this in Chapter 6, but as a reminder:

In my hospital dataset, the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values). Google says that for the Flu, the average person with the disease spreads it to 1-2 people. As our negative binomial distribution only counts the number of additional people each person infects after the first, then setting $E[X] = 0.5$ feels fair. In other words, for the flu, it reasonable

to stay the average person in a hospital with a disease infects 0.5 extra people. That would means that $\mathbb{E}[X] = \frac{r(1-p)}{p} = 0.5$.

I isolated for p in Chapter 6, and got that $p = 0.6$:

Let's say our null hypothesis is H_0: $p = 0.6$. Now, we are curious if maybe COVID has a different degree of infectiousness than the flu (not considering other considerations like how my dataset is limited to people only already at the hospital). This would mean that $E[X] \neq 0.5$, and $p \neq 0.6$.

So, this gives us:

H_0: $p = 0.6$

H_A: $p \neq 0.6$

As we mentioned above, for this type of hypothesis test, we try to maximize the ratio:

$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}$

We can write this out as:

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{\prod_{i=1}^{n} \binom{x_i+r-1}{x} p^r (1-p)^{x_i}}{\prod_{i=1}^{n} \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}}$$

We know that for the numerator, the p that gives the greatest likelihood is $\hat{p}_{MLE}$. In Chapter 1, I calculate the $\hat{p}_{MLE}$ for multiple samples of the Negative Binomial distribution is $\frac{r}{r+\bar{x}}$.

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{\prod_{i=1}^{n} \binom{x_i+r-1}{x} (\frac{r}{r+\bar{x}})^r (1-\frac{r}{r+\bar{x}})^{x_i}}{\prod_{i=1}^{n} \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}}$$

We can simplify this to:

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{\prod_{i=1}^{n} \binom{x_i+r-1}{x} (\frac{r}{r+\bar{x}})^r (1-\frac{r}{r+\bar{x}})^{x_i}}{\prod_{i=1}^{n} \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}}$$

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{(\frac{r}{r+\bar{x}})^{nr} (1-\frac{r}{r+\bar{x}})^{\sum_{i=1}^{n} x_i}}{(p_0)^{nr} (1-p_0)^{\sum_{i=1}^{n} x_i}}$$

We want to reject when $\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} \geq c$, where:

$$P(\texttt{test\_statistic} \geq c) = \alpha = 0.05$$

However, this does not simplify to a distribution I recognize, so I cannot solve for c.

Still, we want to maximize this function. As log is monotonic, we can take the logarithm of the fraction.

$$\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = \log\left(\frac{(\frac{r}{r+\bar{x}})^{nr}(1-\frac{r}{r+\bar{x}})^{\sum_{i=1}^n x_i}}{(p_0)^{nr}(1-p_0)^{\sum_{i=1}^n x_i}}\right)$$

$$\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr\log(\frac{r}{r+\bar{x}}) + \sum_{i=1}^n x_i \log(1-\frac{r}{r+\bar{x}}) - \left(nr\log(p_0) + \sum_{i=1}^n x_i \log(1-p_0)\right)$$

$$\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr\log\left(\frac{\frac{r}{r+\bar{x}}}{p_0}\right) + \sum_{i=1}^n x_i \log\left(\frac{1-\frac{r}{r+\bar{x}}}{1-p_0}\right)$$

$$\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr\log\left(\frac{\frac{r}{r+\bar{x}}}{p_0}\right) + \sum_{i=1}^n x_i \log\left(\frac{1-\frac{r}{r+\bar{x}}}{1-p_0}\right)$$

Once again, this also does not simplify to a distribution I recognize. So, we are left to either simulate or take the asymptotic distribution.

7.2.1 Simulation

Let's simulate many times under the null hypothesis, and calculate $\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}$ each time. For brevity, I will refer to $\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}$ as Λ. Then, as we want to keep $\alpha \leq 0.05$, we would reject if Λ for our data is greater than the 95th percentile of the simulated Λs.

Remember: $\Lambda = \frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{(\frac{r}{r+\bar{x}})^{nr}(1-\frac{r}{r+\bar{x}})^{\sum_{i=1}^n x_i}}{(p_0)^{nr}(1-p_0)^{\sum_{i=1}^n x_i}}$.

However, to prevent underflow, I will take the logarithm.

$\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr\log\left(\frac{\frac{r}{r+\bar{x}}}{p_0}\right) + \sum_{i=1}^n x_i \log\left(\frac{1-\frac{r}{r+\bar{x}}}{1-p_0}\right)$

```
n = length(x)
r = 0.75
p_0 = 0.6
n_sims = 10000

sims <- replicate(n_sims, {
    data <- rnbinom(n = n, size = r, prob = p_0)

    data_bar = mean(data)
    data_sum = sum(data)

    p_mle = r / (r + data_bar)
```

```
    first_term = n * r * log(p_mle / p_0)
    second_term = data_sum * log((1 - p_mle) / (1 - p_0))

    log_lambda = first_term + second_term
    log_lambda
}
)
quantile(sims, 0.95)
```

So, our test statistic is $\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right)$ and our c is about 1.933, depending on the simulation. So, if Λ for my data is above 1.933, we reject H_0.

So, let's calculate $\log(\Lambda)$, while setting $r = 0.75$.

```
n = length(x)
r = 0.75
p_0 = 0.6

x_bar = mean(x)
x_sum = sum(x)

p_mle = r / (r + x_bar)

first_term = n * r * log(p_mle / p_0)
second_term = x_sum * log((1 - p_mle) / (1 - p_0))

log_lambda = first_term + second_term
log_lambda
```

As $\log(\Lambda)$ is 23.55, which is $> c$, we reject H_0.

A power curve stricly from simulation is generated in the code block below.

Note, for the sake of runtime, I lowered `n` to 50 in the code. I recommend messing around and changing `n` to see how the power curve changes.

```
# I encourage you to change n to see how this affects the power curve
n <- 50

p_0 <- 0.6
r = 0.75
n_sims <- 5000
```

```
# Simulating under the null hypothesis to find the 95th quartile (our c value)
null_sims <- replicate(n_sims, {
  data <- rnbinom(n = n, size = r, prob = p_0)
  data_bar <- mean(data)

  p_mle <- r / (r + data_bar)

  # Avoid any potential log(0) errors
  if (p_mle == 1) {
    p_mle <- 0.9999
  }

  # Log-likelihood ratio logic
  log_lambda <- n * r * log(p_mle / p_0) + sum(data) * log((1 - p_mle) / (1 - p_0))
  log_lambda
})
raw_crit_val <- quantile(null_sims, 0.95)

# Set up parameters for power curve
p_alt_space <- seq(0.01, 1, by = 0.05)
powers <- c()

# Simulate alternative hypothesis / null hypothesis and keep track for each one whether we sh

for (p_alt in p_alt_space) {

  rejections <- replicate(n_sims, {

    data_alt <- rnbinom(n = n, size = r, prob = p_alt)
    data_bar_alt <- mean(data_alt)

    p_mle_alt <- r / (r + data_bar_alt)

    # Avoid any potential log(0) errors
    if (p_mle_alt == 1) {
      p_mle_alt <- 0.9999
    }

    # Calculate log_lambda
    log_lambda_alt <- n * r * log(p_mle_alt / p_0) + sum(data_alt) * log((1 - p_mle_alt) / (

    # Check if we should reject?
```

```
    log_lambda_alt > raw_crit_val
  })

  # Append the mean rejection rate (power) for this specific p_alt
  powers <- c(powers, mean(rejections))
}

# Plot results
plot(p_alt_space, powers, type = "l", lwd = 3, col = "#2c3e50",
     xlab = expression(p), ylab = "Power",
     main = "Empirical GLRT Power Curve", ylim = c(0, 1))

abline(v = p_0, col = "#e74c3c", lty = 2)
abline(h = 0.05, col = "gray50", lty = 3)
```

I recommend going back in the code and seeing how changing `n` can effect your power curve.

7.2.2 Asymptotic

We learned in class that under Wilks' Theorem, when only solving for only one parameter and setting all the potential parameters as fixed, then for large n:

$2\log(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}) \sim \chi^2_1$.

Remember, we want to reject when $2\log(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)})$ is large.

So, we set up our rejection bound as:

$2\log(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}) \geq c$, where c the 95% percentile of the chi-squared distribution.

So, let's figure out our c:

We can run this in code:

```
alpha = 0.05
p = 0.6
c = qchisq(1-alpha, 1)
c
```

So, from the code block above, $c = 3.841459$

Now, let's check our data. Let's assume $r = 0.75$. Remember that Wilks' Theorem states that for large n when there is only one unknown parameter.

$2\log(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}) \sim \chi^2_1$.

For our Negative Binomial: $\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr\log\left(\frac{\frac{r}{r+\bar{x}}}{p_0}\right) + \sum_{i=1}^{n} x_i \log\left(\frac{1-\frac{r}{r+\bar{x}}}{1-p_0}\right)$

Let's proceed with $r = 0.75$.

```
n = length(x)
r = 0.75
x_sum = sum(x)
x_bar = mean(x)
p_0 = 0.6
p_mle = r / (r + x_bar)
log_part = n * r * log(p_mle / p_0) + x_sum * log((1 - p_mle) / (1 - p_0))
test_stat = 2 * log_part
test_stat
```

So, we can see from the code block that our dataset's test statistic was 47.09489. If I just plug in the PMFs, I get the same value:

```
log_L_mle  <- dnbinom(x_sum, size = n * r, prob = p_mle, log = TRUE)
log_L_null <- dnbinom(x_sum, size = n * r, prob = p_0, log = TRUE)

test_stat_2 = 2 * (log_L_mle - log_L_null)
test_stat_2
```

As $47.1 > c$ (which was 3.841459), for my COVID hospital dataset, we reject H_0.

This implies that we reject the hypothesis that COVID has the same contagious level as the flu (though this dataset is not perfect as it only consists of people already in the hospital who infected at least 1 person, so I do not want to extrapolate too much). Another way to say this is that we accept the hypothesis that COVID does not have the same contagious level as the flu.

A power curve using Wilks' Theorem would be:

```
# Feel free to change this parameter (n) to see how it affects the curve
n = length(x)

p_alt <- seq(0, 1, by = 0.001)
alpha <- 0.05
crit_val <- qchisq(alpha, df = 1, lower.tail = FALSE)

lambda <- 2 * n * r * (log(p_alt / p_0) + ((1 - p_alt) / p_alt) * log((1 - p_alt) / (1 - p_0
```

```
lambda[p_alt == p_0] <- 0

power_curve <- pchisq(crit_val, df = 1, ncp = lambda, lower.tail = FALSE)

plot(p_alt, power_curve, type = "l", col = "#2c3e50", lwd = 3,
     xlab = expression(textstyle("p")),
     ylab = "Power",
     main = "GLR Power Curve (Wilks' Theorem)",
     ylim = c(0, 1), panel.first = grid(lty = "dotted", col = "gray70"))

abline(v = p_0, col = "#e74c3c", lty = 2, lwd = 2)   # Null Hypothesis baseline

legend("bottomright",
       legend = c("Power Curve", "Null Hypothesis (p = 0.6)"),
       col = c("#2c3e50", "#e74c3c", "#27ae60"),
       cex=0.6,
       lty = c(1, 2, 3), lwd = c(3, 2, 2), bg = "white")
```

Again, I recommend going back in the code and seeing how changing `n` can effect your power curve.

8 Bayesian Hypothesis Testing

8.1 Background

In the previous two chapters, we have talked about frequentist hypothesis testing. We now turn our heads to Bayesian hypothesis testing. For Bayesian hypothesis testing, we can compare the posterior likelihoods of H_0 and H_A. This is called the Bayes Factor. If the Bayes Factor is above 20, then we are considered to have "strong evidence" to reject the null per Kass and Raftery (1995).

For Bayesian hypothesis testing, we are not concerned with limiting Type I error. This is because Type I and Type II errors are frequentist concepts, since they are defined in terms of running the experiment an infinite number of times. That is not relevant to a Bayesian. Instead, Bayesians focus on the posterior probabilities of the null and alternative hypotheses.

Similar to our previous Bayesian metrics, we first need some priors. There are three priors we will need to come up with. The first two are the prior likelihoods of H_0 and H_A, which naturally need to sum to 1. The third is a prior distribution for θ.

Bayesian and frequentist hypothesis testing do not always give the same conclusion. Despite having the same data, the frequentist hypothesis test may reject the null while the Bayesian test accepts. This is known as Lindley's Paradox.

8.2 Bayesian Hypothesis Test

First, let's bring my hospital dataset into R.

```
dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
```

I mention this in Chapter 6, but as a reminder:

In my hospital dataset, the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values). Google says that for the Flu, the average person with the disease spreads it to 1-2 people. As

our negative binomial distribution only counts the number of additional people each person infects after the first, then setting $E[X] = 0.5$ feels fair. In other words, for the flu, it reasonable to stay the average person in a hospital with a disease infects 0.5 extra people. That would means that $\mathbb{E}[X] = \frac{r(1-p)}{p} = 0.5$.

I isolated for p in Chapter 6, and got that $p = 0.6$:

Let's say our null hypothesis is H_0: $p = 0.6$. Now, we are curious if maybe COVID has a different degree of infectiousness than the flu (not considering other considerations like how my dataset is limited to people only already at the hospital). This would mean that $E[X] \neq 0.5$, and and $p \neq 0.6$.

So, this gives us:

H_0: $p = 0.6$

H_A: $p \neq 0.6$

Now, for Bayesian Hypothesis Testing, we need to assign prior probabilities to both the null hypothesis and the alternative hypothesis. As I do not have a great intuition, let's follow the Rule of Insufficient Reason, as described in Efron (2026), and say both are 0.5.

So, we have:

$P(H_0) = 0.5$

$P(H_A) = 0.5$

Next, we need to find the prior distribution for p in H_A. Again, as I do not have a great intuition, let's once again follow the Rule of Insufficient Reason and say $p \sim \text{Uniform}(0, 1)$, as p could be anything from 0 to 1.

So, using Bayes' Rule, this means that:

$P(H_A|x) = \frac{P(H_A)f(x|H_A)}{f(x)}$

$P(H_0|x) = \frac{P(H_0)f(x|H_0)}{f(x)}$

NOTE: In the formula below, I will refer to p in H_0 as p_0. I know we set this equal to 0.6, but for clarity, I think it is easier to understand with p_0.

Remember that $f(x_1, x_2, ...x_n)$ is the product of Negative Binomial random variables.

When taking their ratio, we get the Bayes Factor, which is:

$$
\begin{aligned}
\frac{P(H_A|x_1,...,x_n)}{P(H_0|x_1,...,x_n)} &= \frac{\frac{P(H_A)f(x_1,...,x_n|H_A)}{f(x_1,...,x_n)}}{\frac{P(H_0)f(x_1,...,x_n|H_0)}{f(x_1,...,x_n)}} \\
&= \frac{P(H_A)f(x_1,...,x_n|H_A)}{P(H_0)f(x_1,...,x_n|H_0)} \\
&= \frac{P(H_A)}{P(H_0)}\frac{f(x_1,...,x_n|H_A)}{f(x_1,...,x_n|H_0)} \\
&= \frac{0.5}{0.5}\frac{\int_0^1 f(p)f(x_1,...,x_n|p)dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i}(p_0)^r(1-p_0)^{x_i}} \\
&= \frac{\int_0^1 f(x_1,...,x_n|p)dp}{\prod_{i=1}^n \binom{x+r-1}{x}(p_0)^r(1-p_0)^{x_i}} \\
&= \frac{\int_0^1 \prod_{i=1}^n \binom{x_i+r-1}{x_i}p^r(1-p)^{x_i}dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i}(p_0)^r(1-p_0)^{x_i}} \\
&= \frac{\int_0^1 \prod_{i=1}^n \binom{x_i+r-1}{x_i}p^r(1-p)^{x_i}\,dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i}(p_0)^r(1-p_0)^{x_i}} \\
&= \frac{\int_0^1 \left(\prod_{i=1}^n \binom{x_i+r-1}{x_i}\right)p^{rn}(1-p)^{\sum_{i=1}^n x_i}\,dp}{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i}\right)(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= \frac{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i}\right)\int_0^1 p^{rn}(1-p)^{\sum_{i=1}^n x_i}\,dp}{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i}\right)(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= \frac{\int_0^1 p^{rn}(1-p)^{\sum_{i=1}^n x_i}\,dp}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}}
\end{aligned}
$$

We can simplify the numerator by recognizing that it has the meat of the Beta PDF over its domain.

$$
\begin{aligned}
\frac{P(H_A|x)}{P(H_0|x)} &= \frac{\int_0^1 p^{rn}(1-p)^{\sum_{i=1}^n x_i}\,dp}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= \frac{\frac{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}{\Gamma(rn+1+\sum_{i=1}^n x_i+1)}\int_0^1 \frac{\Gamma(rn+1+\sum_{i=1}^n x_i+1)}{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}p^{rn}(1-p)^{\sum_{i=1}^n x_i}\,dp}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= \frac{\frac{\Gamma(rn+1)\Gamma\left(\sum_{i=1}^n x_i+1\right)}{\Gamma\left(rn+\sum_{i=1}^n x_i+2\right)}}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}}
\end{aligned}
$$

In my hospital dataset, the data seems to fit a negative binomial distribution in which r is not necessarily an integer, meaning we cannot simplify this further with factorials.

Instead, let's plug our data this equation to find the Bayes Factor. Let's say $r = 0.75$, the same r value I used in Chapter 7.

Unfortunately, the denominator gets so small that the code thinks we are dividing by 0. So, let's instead take e to the power of [the log of the numerator subtracted by the *log* of the denominator].

$$\begin{aligned}
\frac{P(H_A|x)}{P(H_0|x)} &= \frac{\frac{\Gamma(rn+1)\Gamma\left(\sum_{i=1}^{n} x_i+1\right)}{\Gamma\left(rn+\sum_{i=1}^{n} x_i+2\right)}}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^{n} x_i}} \\
&= e^{\log\left(\frac{\frac{\Gamma(rn+1)\Gamma\left(\sum_{i=1}^{n} x_i+1\right)}{\Gamma\left(rn+\sum_{i=1}^{n} x_i+2\right)}}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^{n} x_i}}\right)} \\
&= e^{\log\left(\frac{\Gamma(rn+1)\Gamma\left(\sum_{i=1}^{n} x_i+1\right)}{\Gamma\left(rn+\sum_{i=1}^{n} x_i+2\right)}\right)-\log\left((p_0)^{rn}(1-p_0)^{\sum_{i=1}^{n} x_i}\right)} \\
&= e^{\log\left(\Gamma(rn+1)\right)+\log\left(\Gamma\left(\sum_{i=1}^{n} x_i+1\right)\right)-\log\left(\Gamma\left(rn+\sum_{i=1}^{n} x_i+2\right)\right)-\left(rn\log(p_0)+\sum_{i=1}^{n} x_i\log(1-p_0)\right)}
\end{aligned}$$

```
r = 0.75
sum_x = sum(x)
n = length(x)
p_0 = 0.6

first_term = lgamma(r * n+1)
second_term = lgamma(sum_x + 1)
third_term = lgamma(r * n + sum_x + 2)
fourth_term = r * n * log(p_0) + sum_x * log(1-p_0)

exponent = first_term + second_term - third_term - fourth_term

bayes_factor = exp(exponent)
bayes_factor
```

My Bayes Factor is 607815364, meaning there is very strong evidence to reject the null hypothesis.

In Chapter 7, using Frequentist Hypothesis Testing methods, I also rejected the null hypothesis. So, both methods reached the same conclusion, in that we rejected H_0 (we reject the hypothesis that COVID has the same contagious level as the flu). As my Bayesian and frequenist hypothesis tests both come to the same conclusion, I did not encounter Lindley's paradox.

9 Sufficency

9.1 Background

We have finally finished covering both frequentist and Bayesian hypothesis testing. This means we moved onto the final topic of this book: sufficiency.

A sufficient statistic $T(x_1, ..., x_n)$, means that T contains all useful information with respect to the data. This means that once we have T, we do not care how we got there. As a quick example, let's say that $T = \sum_{i=1}^{n} x_i$ and it is a sufficient statistic for θ. This means that for properly estimating θ, we do not need to know $x_1, ..., x_n$. In fact, we only need to know T.

Furthermore, the Rao-Blackwell theorem says that for any unbiased estimator $\hat{\theta}$, then $\hat{\theta}_{RB} = \mathbb{E}[\hat{\theta}(T)]$ is also unbiased and $\text{Var}(\hat{\theta}_{RB}) \leq \text{Var}(\hat{\theta})$. So, this means that the best unbiased estimator only references the data with respect to the sufficient statistic T.

A quick proof is below. First, let's show that if $\hat{\theta}$ is unbiased, then so is $\hat{\theta}_{RB}$.

$$\mathbb{E}[\hat{\theta}_{RB}] = \mathbb{E}[\mathbb{E}[\hat{\theta}|T]]$$
$$\mathbb{E}[\hat{\theta}_{RB}] = \mathbb{E}[\hat{\theta}] \text{ [NOTE: From the Law of Total Exectation]}$$

So, this means that the expected value of both estimators is the same, so they have the same bias. Next, let's show the variance part.

$$\text{Var}(\hat{\theta}) = \mathbb{E}[(\text{Var}(\hat{\theta}|T)] + \text{Var}(\mathbb{E}[\hat{\theta}|T]) \text{ [NOTE: From the Law of Total Variance]}$$
$$\text{Var}(\hat{\theta}) = (\text{value} \geq 0) + \text{Var}(\hat{\theta}_{RB})$$

So, we can see $\text{Var}(\hat{\theta}_{RB}) \leq \text{Var}(\hat{\theta})$.

This gives way to the sufficiency principle: "If you have a sufficient statistic, use it."

The MLE and Posterior Mean are always both functions of a sufficient statistic. In general, our estimators are usually already functions of sufficient statistics, meaning we do not usually have to apply the Rao-Blackwell theorem to improve our estimator (this process is called Rao-Blackwellization).

9.2 Sufficient Statistic

In class, we learned that for data $X_i, ...X_n$ and parameter θ, a statistic $T(x_i, ...x_n)$ is sufficient if either of the following hold:

1: $f_\theta(x_i, ...x_n) = g\Big(\theta, T(x_1, ..., x_n)\Big) \cdot h(x_1, , ...x_n)$,

where:

$g\Big(\theta, T(x_1, ..., x_n)\Big)$ depends on θ only through T

$h(x_1, , ...x_n)$ depends only on $(x_1, , ...x_n)$ and not on θ

2. $P(X_i = x_1, ..., X_n = x_n | T = t)$ does not depend on θ.

For my Negative Binomial dataset, p is the parameter we are interested in estimating, so p will be our θ.

I will show that for my distribution, both definitions hold:

9.2.1 First Proof

The PMF of my negative Binomial Distribution: $X_1, ..., X_n \sim \text{NegBin}(r, p)$ is:

$$f_p(x_1, ..., x_n) = \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i} p^r (1-p)^{x_i}$$

We can simplify this into:

$$\begin{aligned} f_p(x_1, ..., x_n) &= \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i} p^r (1-p)^{x_i} \\ &= p^{rn} \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i} (1-p)^{x_i} \\ &= p^{rn} \left(\prod_{i=1}^{n} \binom{x_i + r - 1}{x_i} \right) (1-p)^{\sum_{i=1}^{n} x_i} \end{aligned}$$

We can re-write this as:

$$f_p(x_1, ..., x_n) = p^{rn} (1-p)^{\sum_{i=1}^{n} x_i} \cdot \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i}$$

Now, as we have been doing, let's assume r is fixed and p is the parameter we are interested in estimating.

Here, if we set $T(X_1, ...x_n) = \sum_{i=1}^{n} x_i$, then we can rewrite the equation as:

$$\begin{aligned} f_p(x_1, ..., x_n) &= p^{rn}(1-p)^{\sum_{i=1}^{n} x_i} \cdot \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i} \\ &= p^{rn}(1-p)^{T} \cdot \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i} \\ &= g\Big(p, T(x_1, ..., x_n)\Big) \cdot h(x_1, , ...x_n) \end{aligned}$$

where:

1. $g\Big(p, T(x_1, ..., x_n)\Big) = p^{rn}(1-p)^{T}$
2. $h(x_1, , ...x_n) = \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i}$

As we were able to write our PDF in terms of $g\Big(p, T(x_1, ..., x_n)\Big) \cdot h(x_1, , ...x_n)$, then this means $T(x_1, ..., x_n)$ is a sufficient statistic.

So, for the Negative Binomial distribution, $\sum_{i=1}^{n} x_i$ is a sufficient statistic.

9.2.2 Second Proof

Similarly, we could have also confirmed this using the other definition we learned in class, that $P(X_1 = x_1, ..., X_n = x_n | T = t)$ does not depend on p.

$$P(X_1 = x_1, ..., Xn = x_n | T = t) = \frac{P(X_1 = x_1, ..., X_n = x_n, T = t)}{P(T = t)}$$

T is the sum of i.i.d. NegBin(r, p) random variables. So, T is a NegBin(nr, p) random variable.

So, we get:

$$\begin{aligned} P(X_1 = x_1, ..., Xn = x_n | T = t) &= \frac{P(X_1 = x_1, ..., X_n = x_n, T = t)}{P(T = t)} \\ P(X_1 = x_1, ..., Xn = x_n | T = t) &= \frac{p^{rn}(1-p)^{t} \cdot \prod_{i=1}^{n} \binom{x_i + r - 1}{x_i}}{\binom{t + nr - 1}{t} p^{rn}(1-p)^{t}} \\ P(X_1 = x_1, ..., Xn = x_n | T = t) &= \frac{\prod_{i=1}^{n} \binom{x_i + r - 1}{x_i}}{\binom{t + nr - 1}{t}} \end{aligned}$$

As we can see, this does not depend on p, proving once again that $T(x_1, ...x_n) = \sum_{i=1}^{n} x_i$ is a sufficient statistic.

9.3 Seeing This in My MLE and Posterior Mean

As I mentioned earlier, the MLE and Posterior Mean are both always functions of a Sufficient Statistic. From Chapter 1, we have that:

$$\hat{p}_{MLE} = \frac{r}{r + \bar{x}}$$
$$\hat{p}_{PM} = \frac{1 + nr}{2 + nr + \sum_{i=1}^{n} x_i}$$

As $T(x_1, ...x_n) = \sum_{i=1}^{n} x_i$, we can rewrite our MLE and Posterior Mean as functions of T:

$$\hat{p}_{MLE} = \frac{r}{r + \frac{T}{n}}$$
$$\hat{p}_{posterior} = \frac{1 + nr}{2 + nr + T}$$

Efron, Bradley. 2026. *To Think Like a Statistician*. Princeton University Press.

Kass, Robert E., and Adrian E. Raftery. 1995. "Bayes Factors." *Journal of the American Statistical Association* 90 (430): 773–95. https://doi.org/10.1080/01621459.1995.10476572.

Lloyd-Smith, Jamie O., Sebastian J. Schreiber, P. Ekkehard Kopp, and Wayne M. Getz. 2005. "Superspreading and the Effect of Individual Variation on Disease Emergence." *Nature* 438 (7066): 355–59. https://doi.org/10.1038/nature04153.